Infrrd // In collaboration with Dayananda Sagar University

Infr[x]

HACK. CODE. SHIP.

[JUN 20, 2026 // DSU MAIN CAMPUS, KANAKAPURA ROAD]

THEME

2,000 Pages, One Decision

Build-Round Problem Statements

Mortgage Document Intelligence

A mortgage loan file can run to two thousand pages. Buried in it is the handful of facts that decide one thing: should this loan move forward? That gap, from a mountain of pages to a single sound decision, is this year's theme, and the two problems below are different ways to close it. **Your team picks one.** You build a working prototype and present it.

Every problem is drawn from a real loan-origination pain point at Infrrd. We care about **something that runs**, not slides about something that might. Read the whole brief before you choose; the problem that wins is not the most ambitious one, but the one a team builds end to end and demonstrates with conviction.

2	3	5	1
PROBLEMS / PICK ONE	PER TEAM	MIN DEMO	WORKING PROTOTYPE

PROBLEM A

Cross-Document QA over a 100+ Page Loan File

Suited to a strong agentic-AI team

PROBLEM B

Structuring the 2,000-Page File: Tables & Logical Pagination

Suited to a systems / ML team

[01] // READ THIS FIRST

How the build round works

This is the build round. You are past screening. The goal now is a **working prototype**, not a plan for one. Pick one of the two problem statements in this brief and build it. How you approach it is entirely up to you.

What you submit

DELIVERABLE	WHAT IT IS
Working prototype	A running application in a real Git repository we can clone, with a short `README` giving exact run steps and your dependencies. **Commit regularly as you build** so your history shows the work taking shape; a single dump at the end is a red flag.
Live demo	A 5-minute demonstration. Show your system working on a real document, end to end.
Short deck	6 to 8 slides covering your approach, architecture, a walkthrough, and what you'd do next. Submit as PDF.

How you're judged

Each team is evaluated by a panel across the dimensions below, on a simple 1 to 5 scale per dimension. Working software is weighted the most heavily: a prototype that runs and does the job beats a beautiful idea that doesn't.

DIMENSION	WHAT WE LOOK FOR	WEIGHT
Working functionality	Does it run? Does it actually do the job on real inputs?	**30%**
Technical methodology	A sound approach, a sensible architecture, honest evaluation of your own results.	**25%**
Innovation	A genuine insight, not a stack of buzzwords. Something that surprises us.	**15%**
Impact & practicality	Would this actually help a loan-ops team? Does it fit a real workflow?	**15%**
Demo & presentation	Clear, concrete, legible. Could you show it to a customer?	**15%**

You'll be judged holistically. These dimensions are a guide, not a checklist: if an entry doesn't hold up properly on any one of them, we may not treat it as a complete solution. A worthy winner tends to be one that stands on its own across the board, not one that leans on a single strength.

Note: if there is a lack of completion across these rubrics, an entry cannot be accepted as a winning solution.

Resources & rules

- Use **any models, libraries, or tools** you like, open-source or commercial. Just list your dependencies and give us clear run instructions so we can reproduce your build.
- Sourcing your own data is part of the challenge. You are free to use any public datasets you can find, or to construct your own inputs to build and test against.
- At demo time, we may hand your team a fresh document or two to run through your system on the spot. We'll only do this if your prototype is already working well enough to handle it, so build something robust enough to take an input it has never seen.
- **Cite anything you reuse.** Starter code and open models are encouraged; passing off others' work as your own is not.
- Build something that **generalizes** rather than something tuned to a handful of samples. Robustness on unfamiliar inputs is what we're looking for.

[02] // PROBLEM STATEMENT A

Cross-Document QA over a 100+ Page Loan File

Ask the loan file a question. Get an answer you can trace back to the page.

SUITED TO
A strong agentic-AI team

SPIRIT OF IT
Answers over a big file, every one traceable

THE SCENARIO

A mortgage loan package can run past a thousand pages: dozens of document types stacked into one enormous file. An underwriter's questions are easy to ask and painful to answer by hand. Some are simple lookups ("What's the borrower's stated monthly income?"). Others require pulling together facts that live in several different places in the file and comparing them. To complicate matters, big files often contain near-duplicate documents (several paystubs, more than one W-2), and the right answer can depend on knowing which is which. And anything an underwriter relies on has to be **traceable back to its source**, or it doesn't survive an audit.

The kinds of questions it should field

The questions a reviewer throws at a file vary enormously in difficulty. Some are a single glance; others need arithmetic across dozens of pages, or hinge on telling near-identical documents apart. A strong system handles the easy ones reliably and stays honest on the hard ones. A representative spread:

- **Direct lookups.** "What is the note interest rate?" "What loan type is this: conventional, FHA, VA, or USDA?" "What is the total monthly housing payment (PITI)?"
- **Counting and structure.** "How many separate documents are in this file?" "How many paystubs are included?" "On which page does the Closing Disclosure begin?" "How many pages does the single longest document span?"
- **Aggregation across many pages.** "How many transactions are listed across all checking statements combined?" "Sum the ending balances of every checking statement." "What is the single largest deposit found anywhere in the checking statements?"
- **Reasoning across documents.** "Confirm the one loan amount that should be identical across the URLA 1003, Form 1008, Closing Disclosure, and Loan Estimate." "Do the W-2 box 1 wages reconcile with the wages on the Form 1040, and what is the agreed figure?" "What is the down payment, that is, purchase price minus loan amount?"
- **Telling near-duplicates apart.** "There are nine checking statements: what is the ending balance on the most recent one?" "Among the ten paystubs, what year-to-date gross pay does the most recent one report?"
- **Questions the file cannot answer.** "What is the annual flood-insurance premium for the property?" "On what date does the appraiser's license expire?" A good system should say it does not know rather than invent a number.

Why this matters

Large loan files are slow and frustrating to navigate, and reviewers spend a lot of time hunting across pages to confirm a single fact. Fast, trustworthy, traceable answers over a big file would change how that work feels, and how quickly it gets done.

> **WHAT WE'RE ASKING YOU TO DO**
>
> Build a working system that can answer questions about a large, multi-document loan file. It should handle both straightforward lookups and questions that require connecting information across several different documents in the file. For every answer it gives, a reviewer should be able to see **where in the file the answer came from**. You will demonstrate it live and submit your prototype and a short write-up. The design is entirely up to you. A sample dataset that emulates real loans will be provided.

Consider how your system should behave when the file holds conflicting or duplicate documents, and what it should do with a question the file simply can't answer. How you handle those situations, and how you let a reviewer check your answers, is a big part of what makes a system like this trustworthy.

[03] // PROBLEM STATEMENT B

Structuring the 2,000-Page File: Tables & Logical Pagination

Turn one enormous, undifferentiated PDF into structured, machine-usable documents.

SUITED TO
A systems / ML team

SPIRIT OF IT
Give structure to the blob, efficiently

THE SCENARIO

Everything else in this hackathon assumes the loan file already makes sense. It usually doesn't. What actually arrives is a single PDF that can run to two thousand pages: dozens of distinct documents scanned and merged into one continuous stream, with no table of contents and no markers saying where one document ends and the next begins. The things downstream work depends on most are **tables**: bank-statement transaction histories, payment schedules, itemized fees, which often run across several pages, with headers that repeat, totals that interrupt, and columns that drift. Two problems sit at the very bottom of the stack and block everything above them: knowing **which pages belong together as one document**, and turning the **tables on those pages into structured data** a machine can actually use rather than a flat wash of text.

What "logical pagination" means here

Splitting the file is not just noticing that the document *type* changed. It is recovering the exact page span of every individual document instance, even when several documents of the **same type** sit back to back. Picture a borrower who attaches three years of federal tax returns. If each Form 1040 is two pages and they are stacked one after another, your system should not simply report "pages 31 to 36 are Form 1040." It should resolve them into three distinct instances: the first year's 1040 starts on page 31 and ends on page 32, the second year's starts on page 33 and ends on page 34, and the third starts on page 35 and ends on page 36. Each instance should carry its own start page, end page, document type, and, where one exists, a distinguishing attribute such as the tax year that tells the three apart. The same idea applies to several months of bank statements, multiple paystubs, or repeated W-2s in a single file: the goal is a clean list of document instances with precise page boundaries, not a coarse label smeared across a wide range of pages.

Why this matters

This is the foundation the other three problems stand on. You can't reconcile income, answer a question, or triage a file until the file has been broken into the right documents and its tables have kept their rows and columns intact. It is also where brute force gets expensive fast: running the largest models over every one of two thousand pages is slow and costly, which makes **doing this efficiently**, not just accurately, the real challenge.

WHAT WE'RE ASKING YOU TO DO

Build a working system that takes a large, unstructured multi-page PDF and gives it structure in two ways: it should **group the pages into the individual documents they belong to**, identifying each document's type and its exact start and end pages (including telling apart several documents of the same type), and it should **recover the structure of the tables** inside (including tables that span more than one page) so the result is something downstream systems can consume, not just read. You will demonstrate it live on a real multi-page file and submit your prototype and a short write-up. A sample dataset that emulates real loans will be provided.

This problem carries an explicit constraint the others don't: **efficiency is part of the brief.** We are especially interested in approaches that lean on open-source or smaller models, keep token and compute usage modest, and could plausibly run at this scale without an expensive dependence on the largest hosted models, including approaches light enough to imagine running close to where the documents live. Be ready to talk about the cost of your approach at two-thousand-page scale, not just whether it works on ten.

Think about how your system decides that a run of pages is one document rather than two, about what "structure" should even mean for a messy real-world table, and about the trade-offs you are making between accuracy and the time, tokens, and hardware your approach demands. A strong entry will be as clear about its efficiency choices as about its results.

[04] // FOR YOUR SUBMISSION

The deck (6–8 slides)

The prototype is the star; the deck is the supporting cast. Keep it short. These are the slides we read first.

1. **Title & pick.** Team, members, which problem (A or B), and your one-line pitch.
2. **Approach.** Your insight in one sentence, then how you tackled it.
3. **Architecture.** One readable diagram. Boxes, arrows, labels.
4. **Walkthrough.** One real document or question, taken end to end through your system.
5. **How well it works.** Honest evidence from your own testing.
6. **Limits & what's next.** What your system doesn't handle yet, and what you'd build with more time.
7. **Optional:** anything central to your story: a data model, the key screens, a cost or speed consideration.

Things that move you up

- A prototype that actually runs on a real document.
- Clear, honest evidence that it works.
- An architecture you can explain simply.
- Naming your tradeoffs and limits openly.
- A tight scope, done well.
- A demo that tells a crisp before/after story.

Things that move you down

- Slides about a system that isn't actually running.
- Claims with no evidence behind them.
- A twenty-box diagram that explains nothing.
- Tuning to the sample data instead of solving the problem.
- A demo that's all narration, no working software.
- Buzzword density with nothing underneath.

> **ONE LAST THING**
>
> Teams that win this kind of round don't pick the most ambitious idea; they pick a focused one, build it end-to-end in the first stretch of the day, and spend the rest making the demo sing. Pick a document. Make it work. Show us the difference it makes. Good luck.

The Infrrd Hackathon Team